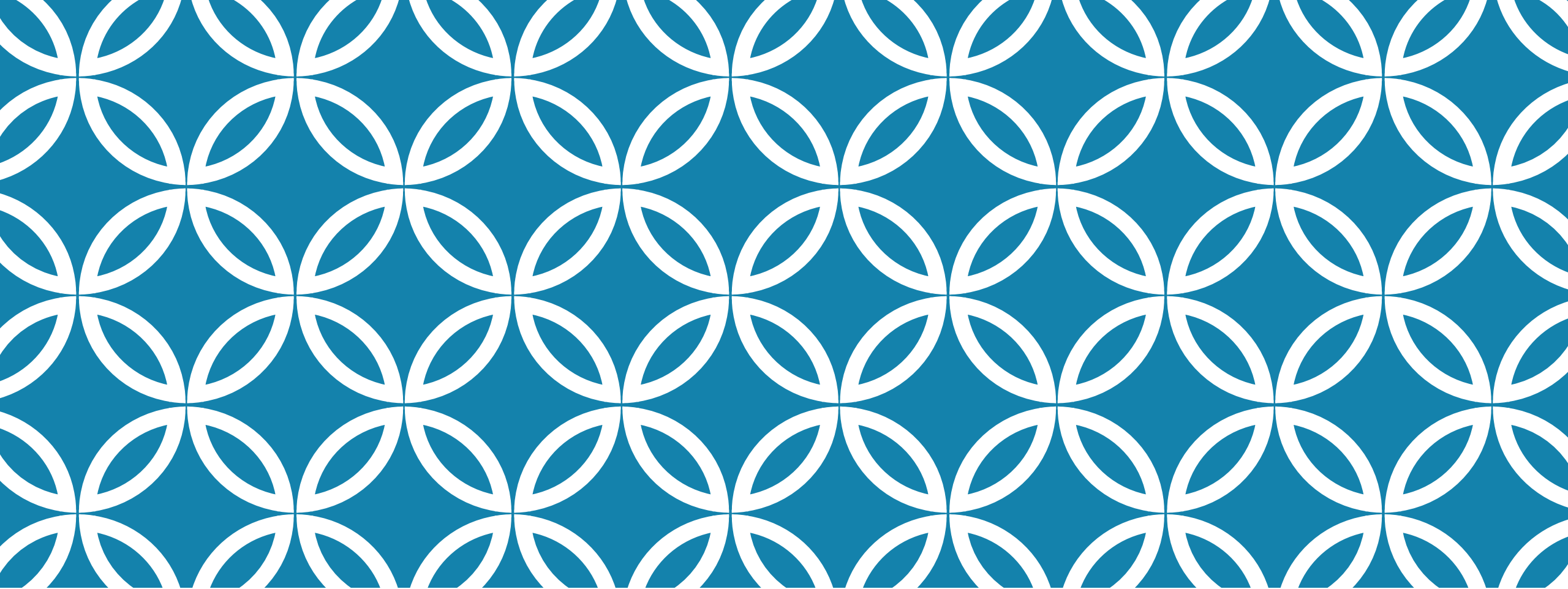




CSE455 - SOFTWARE TESTING

Chapter 7

OUTLINE

Types of Test Tools

Benefits and Risks of Test Automation

Using Test Tools Effectively

Conclusion

TEST TOOLS

Test tools are used to support single or multiple software testing activities

Increase testing efficiency

- Automates repetitive or time-consuming tasks
- **Example:** Static code analysis tools, Automated test execution tools

Increase Test Quality

- Manages large test case repositories
- Helps to identify duplicate, missing test cases

TEST TOOLS

Increase Testing Reliability

- Automated checks provide consistent results
- Used for comparing expected vs. actual, repeatable test sequences

Make Some Tests Possible

- Enables tests not possible manually
- **Example:** Performance testing, Load testing

TYPES OF TEST TOOLS

Test tools vary based on:

- Testing phase
- Automation goals
- Platform/application area

1. Test Management Tools

- Manage, plan and track test cases
- Register and catalog test cases
- Prioritize and evaluate execution results (pass/fail)
- Support for requirement-based testing
- Link test cases to user stories and requirements

TYPES OF TEST TOOLS

2. Requirement Management Tools

- Not strictly test tools, but essential for planning tests
- Save and prioritize requirements
- Track implementation status
- Exchange data with test tools for traceability

3. Defect Management Tools

- Log and manage defect reports
- Track defect status from discovery
- Analyze defects statistically
- Allows custom lifecycle workflows

TYPES OF TEST TOOLS

4. Configuration Management Tools

- Not purely testing tools, but important for version control
- Manage multiple versions of software under test, test data and documentation
- Track which version produced specific test results

5. Continuous Integration Tools

- Used in Agile/Iterative projects
- Automatically integrate code changes
- Trigger integration tests on code check-ins
- Support faster and more reliable builds

TOOL INTEGRATION

Interconnected tool ecosystem – test tools often need to communicate with each other

Important integrations:

- Requirement Management – import/link requirements
- Test Execution Tools – run tests, capture results
- Defect Management – verify defect fixes
- CI Tools – auto-trigger tests during builds
- Configuration Management – track code versions in test cases

TEST REPORTING AND DOCUMENTATION

Generate:

- Test summary reports
- Test schedules
- Test specifications

Helps monitor testing progress and quality

TEST SPECIFICATION TOOLS

To ensure that a test case is reproducible, the individual testing steps, the test data, and the corresponding pre-and post conditions have to be defined and recorded

Test data generator can be used to determine and generate appropriate test data

There are various types of test data generator:

Database-based generators

- Use database structure or contents
- Can also work with file data

Code-based generators

- Analyze source code
- Weak at detecting missing behavior
- White-box technique

TEST SPECIFICATION TOOLS

Interface-based generators

- Analyze APIs or GUIs
- Use boundary value analysis or equivalence class partitioning
- Great for robustness testing

Specification based generators

- Use formal specifications (e.g. UML diagrams)
- Also known as model based testing (MBT)
- Generates test scripts from models

Limitations

- Cannot replace human creativity
- Don't understand what's important or irrelevant, valid expected behavior

STATIC TEST TOOLS

Works on source code, specifications, models or natural language documents

Finds defects early, before runtime, saving time and cost

Review support tools require manual reviews (buddy checks)

For source code, analyze code structure and metric (e.g. cyclomatic complexity) like check for risky code blocks, broken HTML links

DATA FLOW ANALYSIS

Analyze data states through code

Detect anomalies (suspicious usage patterns) – can but doesn't necessarily cause failure

Variable states:

defined (d) – the variable is assigned a value

referenced (r) – the variable's value is referenced

undefined (u) – the variable has no defined value

Anomaly types:

ur – variable used (read) before it's defined

du – defined but never used

dd – defined again without using previous value

DATA FLOW ANALYSIS

```
void Swap (int& Min, int& Max) {  
    int Help;  
    if (Min > Max) {  
        Max = Help;  
        Max = Min;  
        Help = Min;  
    }  
}
```

DATA FLOW ANALYSIS

```
void Swap (int& Min, int& Max) {  
    int Help;  
    if (Min > Max) {  
        Max = Help;    // ur anomaly: Help not initialized  
        Max = Min;     // dd anomaly: Max overwritten before use  
        Help = Min;    // du anomaly: Help value never used  
    }  
}
```

DYNAMIC TEST TOOLS

Automate testing by:

- Feeding test data to the application
- Recording responses
- Logging test results
- Running tests along side the tool can affect results “probe effect”

Unit test framework can aid in integration and unit testing (JUnit)

CAPTURE/REPLAY TOOLS

Also known as test robots

Record UI actions – mouse clicks, keystrokes, UI attributes

Record as a script for later re-use for regression testing

Replay – automatically re-executes script

Compare – expected vs. actual results

GUI object mapping – instead of capturing exact element position in x ,y coordinates, just capture target attributes or path

Structured testing scripting helps with reusability, maintainability, team collaboration

DATA DRIVEN TESTING

Store test data in spreadsheet or databases

Script reads data, run tests, repeats ...

Easy to add test cases

To test against new test data, just add a new entry in spreadsheet

KEYWORD DRIVEN TESTING

Keyword = actions + parameters

Test cases are made using high-level domain terms:

`select (customer)`

`save (contract)`

Keyword mapped to code by test engineers

Limitations

Spreadsheet become complex

Hard to manage dependencies

ADVANCED TOOL SUPPORT

Modern test tools offer:

- Drag & drop keywords
- Database backed test data
- Auto-update of test cases when keywords changes

Comparators perform comparison of expected against actual of test result

Coverage tools measure test completeness (% of logic covered) – requires code instrumentation before test runs

Debuggers help stepping through code, setting breakpoints, watching variables ensuring basic level testing

LOAD AND PERFORMANCE TESTING TOOLS

Test system under high load and ensure performance criteria (e.g. response time) are met

Used for web/cloud based systems, client/server apps, real-time systems

Load testing – simulate many users or transactions in parallel

Performance testing – measure response time, memory usage etc. and highlight bottlenecks or limits

Monitors – track system metrics like CPU, memory, database queries, network traffic

ADDITIONAL TOOL SUPPORT

Security testing – identify vulnerabilities, uses tools like firewall, anti-virus software, penetration testing tools

Usability testing – check for proper accessibility to the target users

Localization – ensure language translation feature working

Portability – ensure cross-platform compatibility

Data quality evaluation tools are used during data migration/conversion

- Ensure data is complete, correct, formatted
- Satisfies business rules
- Critical in replacing legacy systems

BENEFITS AND RISKS OF TEST AUTOMATION

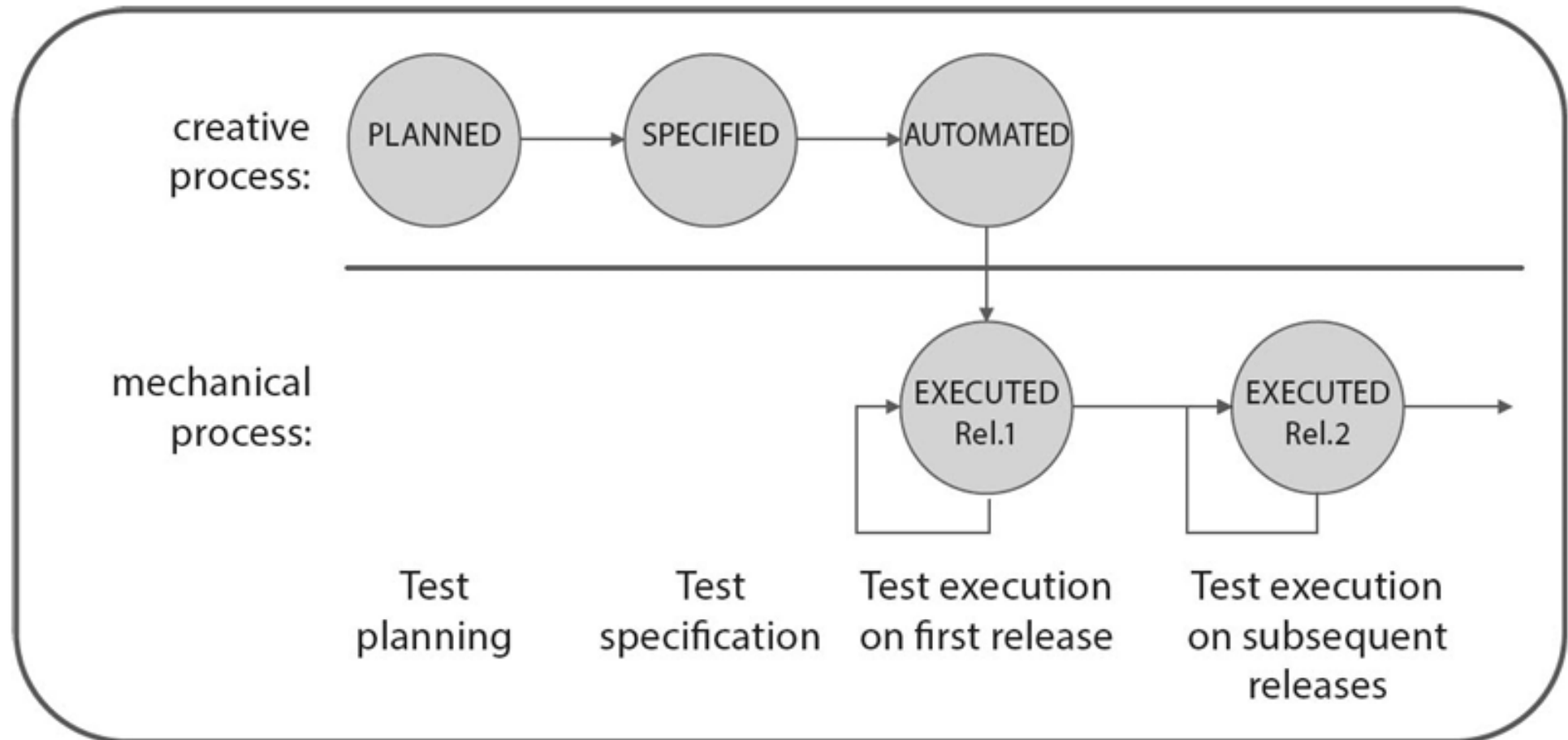
Automation cost

- Software tool purchase and licensing
- Hardware requirement
- Staff training and roll-out

Amortization

- Tools must justify cost over time
- Cost-benefit is negative at first, improves over multiple regression test cycles

TEST CASE LIFE CYCLE



BENEFITS AND RISKS OF TEST AUTOMATION

Cost benefit analysis

- Initial cost > manual test
- Savings begin after some automated runs
- Useful when regression testing is frequent

Improves testing effectiveness

- Finds more failures early
- Reduces long-term support and maintenance cost
- Helps build higher-quality software

BENEFITS AND RISKS OF TEST AUTOMATION

Key advantages

- Supports creative testing by removing repetitive work
- Increases number of test cases without extra effort
- Provides objective data on software quality

Risks of introducing automation

- Unrealistic expectations of tool capabilities
- Tools used where manual testing is more efficient
- Incompatibility with existing tools
- Open sources project discontinued
- Lack of support

USING TOOLS EFFECTIVELY

Start with process, not tools

“Automating chaos just gives faster chaos”

Automate only well-organized test processes

Improve manual testing maturity first

Consider learning curve, productivity may initially drop

Avoid tool introduction during high-stress phases

TOOL SELECTION STEPS

- ❑ Define requirements and priorities
- ❑ Research and compile tool candidates
- ❑ Run demos and trails
- ❑ Build shortlist
- ❑ Conduct a proof of concept

Selection criteria:

- Compatibility with test object
- Staff skill and training required
- Integration with existing tools
- Platform, vendor support, cost, licensing model
- Must mark essential criteria (deal-breakers)

PILOT PROJECT

After selecting tool, validate it in real-world project

Gain practical insights and identify gaps

Should be run by a team not involved in tool selection

Identify training needs

Create re-usable test libraries (if test driver/robots used)

Confirm that useful metrics can be collected and reported

ROLLING OUT THE TOOL

Perform gradual rollout

Embed tools into core processes

Provide training and coaching

Create and share guidelines

Continuously collect feedback and measure tool adoption

Tools are only effective when:

- Properly selected
- Gradually adopted
- Backed by structured processes and support

CONCLUSION

For each phase in the testing process, tools are available to help testers perform their work more efficiently, reliably, or at a higher quality level. Some tasks, such as load and performance testing, can only be performed satisfactorily with tool-based support

Component and integration testing tools are squarely aimed at developers. These include unit test frameworks, test-driven development (TDD) frameworks, continuous integration (CI) tools, static analysis tools, and dynamic analysis (code coverage) tools

Test tools only provide benefits if the testing process itself is mastered and runs in a well-defined manner